

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma Sorting”

disusun oleh: Dennis Shauqi Akbar

2511533020

Dosen pengampu: Dr. Wahyudi. S.T.M.T

Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN

2025

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman. Tugas Individu ini kami susun dengan tujuan untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, dengan Judul “Algortima Sorting”

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pengampu yang telah memberikan arahan sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami.

Padang, 05 Juni 2026

Dennis Shauqi Akbar

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Pengertian Algoritma Sorting	2
2.2 Implementasi Pada Java.....	2
2.2.1 BubbleSortGUI_2511533020	2
2.2.2 MergeSort_2511533020.....	5
2.2.3 MergeSortGUI_2511533020	6
2.2.4 QuickSort_2511533020	9
2.2.5 ShellSort_2511533020.....	10
BAB III PENUTUP	11
3.1 Kesimpulan	11
3.2 Saran	11
3.3 Penutup	11
DAFTAR PUSTAKA.....	iii

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sorting atau pengurutan data merupakan salah satu operasi dasar dalam pemrograman. Terdapat berbagai macam algoritma sorting dengan karakteristik yang berbeda-beda. Pada praktikum ini, dipelajari empat algoritma sorting yaitu Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort. Bubble Sort adalah algoritma sederhana yang bekerja dengan membandingkan dan menukar elemen yang berdekatan. Merge Sort menggunakan pendekatan divide and conquer dengan membagi array menjadi bagian kecil lalu menggabungkannya kembali.

Quick Sort juga menggunakan divide and conquer dengan memilih pivot untuk mempartisi array. Shell Sort adalah perbaikan dari Insertion Sort dengan membandingkan elemen yang berjarak tertentu. Selain implementasi console, terdapat juga program GUI untuk memvisualisasikan Bubble Sort dan Merge Sort langkah demi langkah.

1.2 Tujuan

Tujuan dari praktikum ini adalah memahami cara kerja keempat algoritma sorting tersebut, baik secara teoritis maupun implementasi dalam kode. Diharapkan mampu membedakan mekanisme masing-masing algoritma, serta mengamati visualisasi proses sorting melalui program GUI yang disediakan.

1.3 Manfaat

Manfaat yang diperoleh antara lain pemahaman tentang efisiensi berbagai algoritma sorting, kemampuan mengimplementasikan Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort dalam Java, serta pengalaman membuat program GUI untuk visualisasi sorting. Program GUI yang dihasilkan dapat membantu dalam menjelaskan alur algoritma kepada orang lain karena setiap langkah ditampilkan secara visual.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Algoritma sorting adalah prosedur untuk mengatur sekumpulan data ke dalam urutan tertentu (ascending atau descending). Terdapat banyak algoritma, di antaranya:

- **Bubble Sort:** Membandingkan elemen berdekatan dan menukar jika urutan salah. Proses diulang hingga terurut.
- **Merge Sort:** Membagi array menjadi dua bagian, mengurutkan masing-masing bagian secara rekursif, lalu menggabungkan (merge) kedua bagian yang sudah terurut.
- **Quick Sort:** Memilih pivot, mempartisi array sehingga elemen kiri pivot lebih kecil dan kanan lebih besar, lalu rekursif pada sub-array.
- **Shell Sort:** Memperbaiki Insertion Sort dengan membandingkan elemen yang memiliki jarak (gap) tertentu, jarak diperkecil secara bertahap.

2.2 Implementasi Pada Java

Berikut adalah penjelasan untuk masing-masing kode program.

2.2.1 BubbleSortGUI_2511533020

```
1 package pekan0_2511533020;
2
30 import java.awt.BorderLayout;
21
22 public class BubbleSortGUI_2511533020 extends JFrame {
23
24     private static final long serialVersionUID = 1L;
25
26     private JPanel contentPane_3020;
27     private JTextField inputField_3020;
28     private JButton inputButton_3020;
29     private JButton stepButton_3020;
30     private JButton resetButton_3020;
31     private JPanel panelArray_3020;
32     private JTextArea stepArea_3020;
33
34     private int[] array_3020;
35     private JLabel[] labelArray_3020;
36
37     private int i_3020 = 0;
38     private int j_3020 = 0;
39     private int stepCount_3020 = 1;
40     private boolean sorting_3020 = false;
41
42     public static void main(String[] args) {
43        .EventQueue.invokeLater(() -> {
44             try {
45                 BubbleSortGUI_2511533020 frame = new BubbleSortGUI_2511533020();
46                 frame.setVisible(true);
47             } catch (Exception e) {
48                 e.printStackTrace();
49             }
50         });
51     }
52
53     public BubbleSortGUI_2511533020() {
54         setTitle("Visualisasi Bubble Sort");
55         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```

56     setBounds(100, 100, 900, 900);
57
58     contentPane_3020 = new JPanel();
59     contentPane_3020.setBorder(new EmptyBorder(10, 10, 10, 10));
60     contentPane_3020.setLayout(new BorderLayout(10, 10));
61     setContentPane(contentPane_3020);
62
63     JPanel topPanel = new JPanel(new FlowLayout());
64     JLabel lblInput = new JLabel("Masukkan angka (pisahkan koma):");
65     InputField_3020 = new JTextField(25);
66     InputButton_3020 = new JButton("Tampilkan");
67
68     topPanel.add(lblInput);
69     topPanel.add(InputField_3020);
70     topPanel.add(InputButton_3020);
71     contentPane_3020.add(topPanel, BorderLayout.NORTH);
72
73     panelArray_3020 = new JPanel();
74     panelArray_3020.setLayout(new FlowLayout());
75     contentPane_3020.add(panelArray_3020, BorderLayout.CENTER);
76
77     StepArea_3020 = new JTextArea();
78     StepArea_3020.setEditable(false);
79     JScrollPane scrollPane = new JScrollPane(StepArea_3020);
80     scrollPane.setPreferredSize(new Dimension(300, 0));
81     contentPane_3020.add(scrollPane, BorderLayout.EAST);
82
83     JPanel bottomPanel = new JPanel();
84     StepButton_3020 = new JButton("Sorting Selanjutnya");
85     resetButton_3020 = new JButton("Reset");
86     StepButton_3020.setEnabled(false);
87
88     bottomPanel.add(StepButton_3020);
89     bottomPanel.add(resetButton_3020);
90     contentPane_3020.add(bottomPanel, BorderLayout.SOUTH);
91
92     InputButton_3020.addActionListener(e -> setArrayFromInput_3020());
93     StepButton_3020.addActionListener(e -> performStep_3020()); // -> dipanggil performStep_3020

```

```

94     resetButton_3020.addActionListener(e -> reset_3020());
95
96     private void setArrayFromInput_3020() {
97         String text = InputField_3020.getText().trim();
98         if (text.isEmpty()) return;
99         String[] parts = text.split(",");
100         array_3020 = new int[parts.length];
101
102         try {
103             for (int k_3020 = 0; k_3020 < parts.length; k_3020++) {
104                 array_3020[k_3020] = Integer.parseInt(parts[k_3020].trim());
105             }
106         } catch (NumberFormatException e) {
107             JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
108             return;
109         }
110
111         i_3020 = 0;
112         j_3020 = 0;
113         stepCount_3020 = 0;
114         sorting_3020 = true;
115         StepArea_3020.setEnabled(true);
116         StepArea_3020.setText("");
117
118         panelArray_3020.removeAll();
119         JLabelArray_3020 = new JLabel[array_3020.length];
120
121         for (int k_3020 = 0; k_3020 < array_3020.length; k_3020++) {
122             JLabelArray_3020[k_3020] = new JLabel(String.valueOf(array_3020[k_3020]));
123             JLabelArray_3020[k_3020].setBackgroundColor(Color.PINK);
124             JLabelArray_3020[k_3020].setOpaque(true);
125             JLabelArray_3020[k_3020].setForeground(Color.BLUE);
126             JLabelArray_3020[k_3020].setBorder(BorderFactory.createLineBorder(Color.BLACK));
127             JLabelArray_3020[k_3020].setPreferredSize(new Dimension(50, 30));
128             JLabelArray_3020[k_3020].setHorizontalAlignment(JLabel.CENTER);
129         }

```

```

130         panelArray_3020.add(JLabelArray_3020[k_3020]);
131     }
132
133     panelArray_3020.revalidate();
134     panelArray_3020.repaint();
135
136     private void performStep_3020() {
137         if (sorting_3020 || i_3020 >= array_3020.length - 1) {
138             sorting_3020 = false;
139             StepButton_3020.setEnabled(false);
140             JOptionPane.showMessageDialog(this, "Sorting Selesai!");
141             return;
142         }
143
144         resetHighlights_3020();
145         StringBuilder stepLog = new StringBuilder();
146
147         JLabelArray_3020[i_3020].setBackgroundColor(Color.CYAN);
148         JLabelArray_3020[j_3020 + 1].setBackgroundColor(Color.CYAN);
149
150         if (array_3020[i_3020] > array_3020[j_3020 + 1]) {
151             // Swap
152             int temp_3020 = array_3020[i_3020];
153             array_3020[i_3020] = array_3020[j_3020 + 1];
154             array_3020[j_3020 + 1] = temp_3020;
155
156             updateLabels_3020();
157
158             JLabelArray_3020[i_3020].setBackgroundColor(Color.RED);
159             JLabelArray_3020[j_3020 + 1].setBackgroundColor(Color.RED);
160
161             stepLog.append("Langkah ").append(stepCount_3020).append(": Memukuk elemen ke-")
162                 .append(i_3020).append(" ").append(array_3020[i_3020 + 1]).append(" dengan ke-")
163                 .append(j_3020 + 1).append(" ").append(array_3020[j_3020]).append("\n");
164         } else {
165             stepLog.append("Langkah ").append(stepCount_3020).append(": Tidak ada pertukaran antara ke-")

```

```

166             .append(i_3020).append(" dan ke-").append(j_3020 + 1).append("\n");
167         }
168
169         stepLog.append("Hasil: ").append(arrayToString(array_3020)).append("\n\n");
170         StepArea_3020.append(stepLog.toString()); //
171         j_3020++;
172
173         if (j_3020 >= array_3020.length - i_3020 - 1) {
174             JLabelArray_3020[array_3020.length - i_3020 - 1].setBackgroundColor(Color.GREEN);
175             j_3020 = 0;
176             i_3020++;
177             stepCount_3020++;
178         }
179
180         if (i_3020 >= array_3020.length - 1) {
181             sorting_3020 = false;
182             StepButton_3020.setEnabled(false);
183             JOptionPane.showMessageDialog(this, "Sorting Selesai!");
184         }
185     }
186
187     private void updateLabels_3020() {
188         for (int k_3020 = 0; k_3020 < array_3020.length; k_3020++) {
189             JLabelArray_3020[k_3020].setText(String.valueOf(array_3020[k_3020]));
190         }
191     }
192
193     private void resetHighlights_3020() {
194         if (JLabelArray_3020 == null) return;
195         for (JLabel label : JLabelArray_3020) {
196             label.setBackgroundColor(Color.BLUE);
197         }
198     }
199
200     private void reset_3020() {
201         InputField_3020.setText("");

```

```

208     panelArray_3020.removeAll();
209     panelArray_3020.revalidate();
210     panelArray_3020.repaint();
211     stepArea_3020.setText("");
212     stepButton_3020.setEnabled(false);
213
214     array_3020 = null;
215     labelArray_3020 = null;
216     sorting_3020 = false;
217     i_3020 = 0;
218     j_3020 = 0;
219     stepCount_3020 = 1;
220 }
221
222 private String arrayToString(int[] arr) {
223     StringBuilder sb = new StringBuilder();
224     for (int k_3020 = 0; k_3020 < arr.length; k_3020++) {
225         sb.append(arr[k_3020]);
226         if (k_3020 < arr.length - 1) sb.append(", ");
227     }
228     return sb.toString();
229 }
230 }

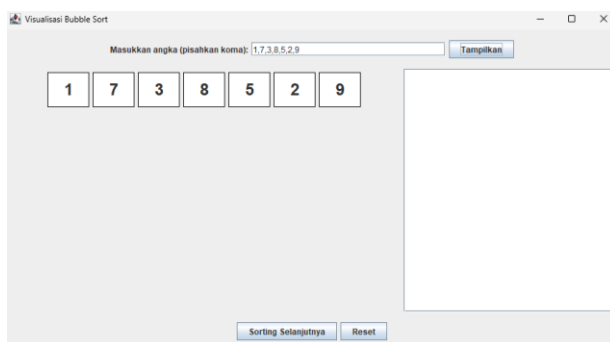
```

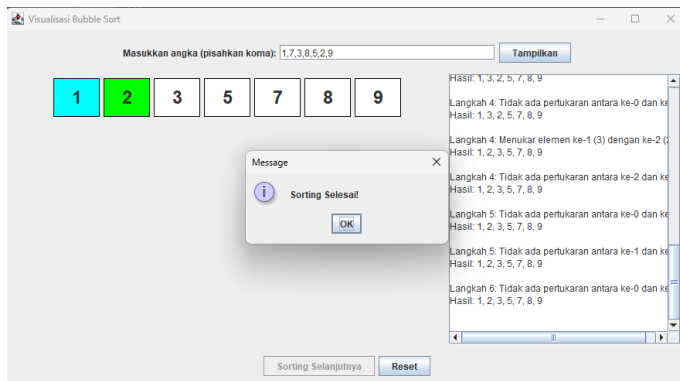
(Gambar 2.2.1.1)

Program ini merupakan GUI untuk memvisualisasikan Bubble Sort langkah demi langkah. Kelas BubbleSortGUI_2511533020 memperluas JFrame. Komponen GUI meliputi JTextField untuk input angka (dipisah koma), JButton “Tampilkan” untuk memproses input, JButton “Sorting Selanjutnya” untuk menjalankan satu langkah sorting, JButton “Reset” untuk membersihkan, JPanel untuk menampilkan array dalam bentuk label berwarna, dan JTextArea untuk log langkah.

Setelah array di-set, variabel i_3020 dan j_3020 diinisialisasi untuk mengontrol perulangan. Method performStep_3020() mengeksekusi satu iterasi dari algoritma Bubble Sort: membandingkan array[j] dan array[j+1], menukar jika perlu, lalu memperbarui tampilan dan log. Warna label diubah saat pembandingan (cyan), saat terjadi pertukaran (merah), dan saat elemen sudah terurut di akhir pass (hijau). Proses berlanjut hingga semua elemen terurut. Method reset_3020() mengosongkan semua data.

Output jika dimasukkan nilai 1,7,3,8,5,2,9





(Gambar 2.2.1.2)

2.2.2 MergeSort_2511533020

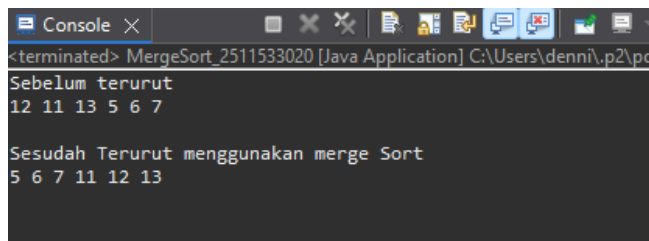
```

1 package pekan8_2511533020;
2
3 public class MergeSort_2511533020 {
4     void merge_3020(int arr_3020[], int l_3020, int m_3020, int r_3020) {
5         int n1_3020 = m_3020 - l_3020 + 1;
6         int n2_3020 = r_3020 - m_3020;
7
8         int L_3020[] = new int[n1_3020];
9         int R_3020[] = new int[n2_3020];
10
11         for (int i_3020 = 0; i_3020 < n1_3020; ++i_3020)
12             L_3020[i_3020] = arr_3020[l_3020 + i_3020];
13         for (int j_3020 = 0; j_3020 < n2_3020; ++j_3020)
14             R_3020[j_3020] = arr_3020[m_3020 + 1 + j_3020];
15
16         int i_3020 = 0, j_3020 = 0;
17         int k_3020 = l_3020;
18
19         while (i_3020 < n1_3020 && j_3020 < n2_3020) {
20             if (L_3020[i_3020] <= R_3020[j_3020]) {
21                 arr_3020[k_3020] = L_3020[i_3020];
22                 i_3020++;
23             } else {
24                 arr_3020[k_3020] = R_3020[j_3020];
25                 j_3020++;
26             }
27             k_3020++;
28         }
29
30         while (i_3020 < n1_3020) {
31             arr_3020[k_3020] = L_3020[i_3020];
32             i_3020++;
33             k_3020++;
34         }
35
36         while (j_3020 < n2_3020) {
37             arr_3020[k_3020] = R_3020[j_3020];
38             j_3020++;
39             k_3020++;
40         }
41     }
42
43     void sort_3020(int arr_3020[], int l_3020, int r_3020) {
44         if (l_3020 < r_3020) {
45             int m_3020 = (l_3020 + r_3020) / 2;
46             sort_3020(arr_3020, l_3020, m_3020);
47             sort_3020(arr_3020, m_3020 + 1, r_3020);
48             merge_3020(arr_3020, l_3020, m_3020, r_3020);
49         }
50     }
51
52     static void printArray_3020(int arr_3020[]) {
53         int n_3020 = arr_3020.length;
54         for (int i_3020 = 0; i_3020 < n_3020; ++i_3020)
55             System.out.print(arr_3020[i_3020] + " ");
56         System.out.println();
57     }
58
59     public static void main(String[] args) {
60         int arr_3020[] = { 12, 11, 13, 5, 6, 7 };
61         System.out.println("Sebelum terurut");
62         printArray_3020(arr_3020);
63         MergeSort_2511533020 ob_3020 = new MergeSort_2511533020();
64         ob_3020.sort_3020(arr_3020, 0, arr_3020.length - 1);
65         System.out.println("\nSetelah Terurut menggunakan merge Sort");
66         printArray_3020(arr_3020);
67     }
68 }

```


(Gambar 2.2.2.1)

Kelas MergeSort_2511533020 mengimplementasikan algoritma Merge Sort secara console. Method merge_3020() menggabungkan dua subarray arr[l..m] dan arr[m+1..r] dengan bantuan array sementara L dan R. Method sort_3020() memanggil dirinya sendiri secara rekursif untuk membagi array, lalu memanggil merge_3020(). Pada method main, array {12,11,13,5,6,7} dicetak sebelum dan sesudah sorting. Output menunjukkan array terurut ascending.



(Gambar 2.2.2.2)

2.2.3 MergeSortGUI_2511533020

```

1 package pekan8_2511533020;
2
3 import java.awt.BorderLayout;
4
5 public class MergeSortGUI_2511533020 extends JFrame {
6     private static final long serialVersionUID = 1L;
7     private int[] array_3020;
8     private JLabel[] labelArray_3020;
9     private JButton stepButton_3020, resetButton_3020, setButton_3020;
10    private JTextField inputField_3020;
11    private JPanel panelArray_3020;
12    private JTextArea stepArea_3020;
13
14    private Queue<int> mergeQueue_3020 = new LinkedList<>();
15    private boolean isMerging_3020 = false;
16    private boolean copying_3020 = false;
17    private int left_3020, mid_3020, right_3020;
18    private int[] temp_3020;
19    private int i_3020, j_3020, k_3020;
20    private int stepCount_3020 = 1;
21
22    public MergeSortGUI_2511533020() {
23        setTitle("Merge Sort Langkah per Langkah");
24        setSize(750, 400);
25        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
26        setLocationRelativeTo(null);
27        setLayout(new BorderLayout());
28
29        JPanel inputPanel = new JPanel(new FlowLayout());
30        inputField_3020 = new JTextField(30);
31        setButton_3020 = new JButton("Set Array");
32        inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
33        inputPanel.add(inputField_3020);
34        inputPanel.add(setButton_3020);
35
36        panelArray_3020 = new JPanel();
37        panelArray_3020.setLayout(new FlowLayout());
38
39        JPanel controlPanel = new JPanel();
40        stepButton_3020 = new JButton("Langkah Selanjutnya");
41        resetButton_3020 = new JButton("Reset");
42        stepButton_3020.setEnabled(false);

```

```

63     controlPanel.add(stepButton_3020);
64     controlPanel.add(resetButton_3020);
65
66     stepArea_3020 = new JTextArea(1, 60);
67     stepArea_3020.setEditable(false);
68     stepArea_3020.setFont(new Font("Monospaced", Font.PLAIN, 14));
69     JScrollPane scrollPane = new JScrollPane(stepArea_3020);
70
71     add(inputPanel, BorderLayout.NORTH);
72     add(panelArray_3020, BorderLayout.CENTER);
73     add(controlPanel, BorderLayout.SOUTH);
74     add(scrollPane, BorderLayout.EAST);
75
76     stepButton_3020.addActionListener(e -> setArrayFromInput_3020());
77     stepButton_3020.addActionListener(e -> performStep_3020());
78     resetButton_3020.addActionListener(e -> reset_3020());
79 }
80
81 private void generateMergeSteps_3020(int l_3020, int r_3020) {
82     if (l_3020 < r_3020) {
83         int m_3020 = (l_3020 + r_3020) / 2;
84         generateMergeSteps_3020(l_3020, m_3020);
85         generateMergeSteps_3020(m_3020 + 1, r_3020);
86         mergeQueue_3020.add(new int[]{l_3020, m_3020, r_3020});
87     }
88 }
89
90 private void setArrayFromInput_3020() {
91     String text = inputField_3020.getText().trim();
92     if (text.isEmpty()) return;
93     String[] parts = text.split(",");
94     array_3020 = new int[parts.length];
95     try {
96         for (int i_3020 = 0; i_3020 < parts.length; i_3020++) {
97             array_3020[i_3020] = Integer.parseInt(parts[i_3020].trim());
98         }
99     } catch (NumberFormatException e) {
100         JOptionPane.showMessageDialog(this, "Masukkan hanya Angka dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
101         return;
102     }
103     labelArray_3020 = new JLabel(array_3020.length);
104     panelArray_3020.removeAll();
105     for (int i_3020 = 0; i_3020 < array_3020.length; i_3020++) {
106         labelArray_3020[i_3020] = new JLabel(String.valueOf(array_3020[i_3020]));

```

```

106         labelArray_3020[i_3020] = new JLabel(String.valueOf(array_3020[i_3020]));
107         labelArray_3020[i_3020].setFont(new Font("Arial", Font.BOLD, 24));
108         labelArray_3020[i_3020].setOpaque(false);
109         labelArray_3020[i_3020].setBackground(Color.WHITE);
110         labelArray_3020[i_3020].setBorder(BorderFactory.createLineBorder(Color.BLACK));
111         labelArray_3020[i_3020].setPreferredSize(new Dimension(50, 30));
112         labelArray_3020[i_3020].setHorizontalAlignment(SwingConstants.CENTER);
113         panelArray_3020.add(labelArray_3020[i_3020]);
114     }
115
116     mergeQueue_3020.clear();
117     generateMergeSteps_3020(0, array_3020.length - 1);
118     stepButton_3020.setEnabled(true);
119     stepArea_3020.setText("");
120     stepCount_3020 = 1;
121     isMerging_3020 = false;
122     panelArray_3020.revalidate();
123     panelArray_3020.repaint();
124 }
125
126 private void performStep_3020() {
127     resetHighlights_3020();
128
129     if (!isMerging_3020 && !mergeQueue_3020.isEmpty()) {
130         int[] range = mergeQueue_3020.poll();
131         left_3020 = range[0];
132         mid_3020 = range[1];
133         right_3020 = range[2];
134         temp_3020 = new int[right_3020 - left_3020 + 1];
135         l_3020 = left_3020;
136         j_3020 = mid_3020 + 1;
137         k_3020 = 0;
138         copying_3020 = false;
139         isMerging_3020 = true;
140         stepArea_3020.append("Langkah " + stepCount_3020++ + ": Mulai merge dari " + left_3020 + " ke " + right_3020 + "\n");
141         return;
142     }
143
144     if (isMerging_3020 && !copying_3020) {
145         if (l_3020 <= mid_3020 && l_3020 <= right_3020) {
146             labelArray_3020[i_3020].setBackground(Color.CYAN);
147             labelArray_3020[j_3020].setBackground(Color.CYAN);
148             if (array_3020[l_3020] < array_3020[j_3020]) {
149                 temp_3020[k_3020++] = array_3020[l_3020++];

```

```

150         } else {
151             temp_3020[k_3020++] = array_3020[j_3020++];
152         }
153         stepArea_3020.append("Langkah " + stepCount_3020++ + ": Bandingkan dan salin elemen\n");
154         return;
155     } else if (l_3020 <= mid_3020) {
156         temp_3020[k_3020++] = array_3020[l_3020++];
157         stepArea_3020.append("Langkah " + stepCount_3020++ + ": Salin sisa kiri\n");
158         return;
159     } else if (l_3020 <= right_3020) {
160         temp_3020[k_3020++] = array_3020[l_3020++];
161         stepArea_3020.append("Langkah " + stepCount_3020++ + ": Salin sisa kanan\n");
162         return;
163     } else {
164         copying_3020 = true;
165         k_3020 = 0;
166         return;
167     }
168 }
169
170 if (copying_3020 && k_3020 < temp_3020.length) {
171     array_3020[left_3020 + k_3020] = temp_3020[k_3020];
172     labelArray_3020[left_3020 + k_3020].setText(String.valueOf(temp_3020[k_3020]));
173     labelArray_3020[left_3020 + k_3020].setBackground(Color.GREEN);
174     k_3020++;
175     stepArea_3020.append("Langkah " + stepCount_3020++ + ": Tempelkan ke array utama\n");
176     return;
177 }
178
179 if (copying_3020 && k_3020 == temp_3020.length) {
180     isMerging_3020 = false;
181     copying_3020 = false;
182 }
183
184 if (mergeQueue_3020.isEmpty() && !isMerging_3020) {
185     stepArea_3020.append("Selesai.\n");
186     stepButton_3020.setEnabled(false);
187     JOptionPane.showMessageDialog(this, "Merge Sort Selesai"); // + Dischalliki
188 }
189
190 private void resetHighlights_3020() { // + Dischalliki: Highlights Jadi Highlights
191     if (labelArray_3020 == null) return;
192     for (JLabel label : labelArray_3020) { // + Dischalliki: JLabel_3020 Jadi JLabel

```

```

194     if (label.getBackground() != Color.GREEN) {
195         label.setBackground(Color.WHITE);
196     }
197 }
198 }
199
200 private void reset_3020() {
201     inputField_3020.setText("");
202     panelArray_3020.removeAll();
203     panelArray_3020.revalidate();
204     panelArray_3020.repaint();
205     stepArea_3020.setText("");
206     stepButton_3020.setEnabled(false);
207     if (mergeQueue_3020 != null) {
208         mergeQueue_3020.clear();
209     }
210     isMerging_3020 = false;
211     copying_3020 = false;
212     stepCount_3020 = 1;
213     array_3020 = null;
214     labelArray_3020 = null;
215 }
216
217 public static void main(String[] args) {
218     SwingUtilities.invokeLater(() -> {
219         MergeSortGUI_2511533020 gui_3020 = new MergeSortGUI_2511533020();
220         gui_3020.setVisible(true);
221     });
222 }
223 }

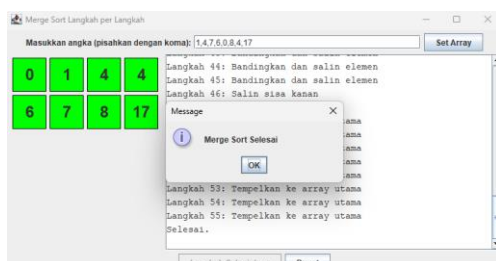
```

(Gambar 2.2.3.1)

Program GUI ini memvisualisasikan Merge Sort langkah demi langkah. Kelas MergeSortGUI_2511533020 memiliki Queue<int[]> mergeQueue_3020 untuk menyimpan langkah-langkah merge yang dihasilkan dari method generateMergeSteps_3020() secara rekursif. Setelah array di-set, queue berisi pasangan {left, mid, right} untuk setiap merge. Method performStep_3020() memproses satu langkah: memulai merge baru, membandingkan elemen kiri dan kanan (ditandai warna cyan), menyalin ke array sementara, lalu menempelkan kembali ke array utama dengan warna hijau. Setiap langkah dicatat di stepArea_3020.

Proses berlanjut sampai queue kosong. Program ini memberikan visualisasi yang jelas tentang proses divide and conquer pada Merge Sort.

Output jika dimasukkan nilai 1,4,7,6,0,8,4,17



(Gambar 2.2.3.2)

2.2.4 QuickSort_2511533020

```

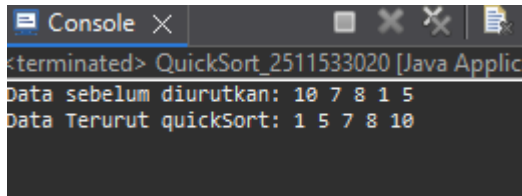
1 package pekang_2511533020;
2
3 public class QuickSort_2511533020 {
4     static void swap(int[] arr_3020, int i_3020, int j_3020)
5     {
6         int temp_3020 = arr_3020[i_3020];
7         arr_3020[i_3020] = arr_3020[j_3020];
8         arr_3020[j_3020] = temp_3020;
9     }
10
11     // Metode pemilihan untuk menentukan pivot menggunakan Median-of-Three
12     static void medianOfThree(int[] arr_3020, int low_3020, int high_3020)
13     {
14         int mid_3020 = low_3020 + (high_3020 - low_3020) / 2;
15
16         // Menukar elemen low, mid, dan high
17         if (arr_3020[low_3020] > arr_3020[mid_3020]) {
18             swap(arr_3020, low_3020, mid_3020);
19         }
20         if (arr_3020[low_3020] > arr_3020[high_3020]) {
21             swap(arr_3020, low_3020, high_3020);
22         }
23         if (arr_3020[mid_3020] > arr_3020[high_3020]) {
24             swap(arr_3020, mid_3020, high_3020);
25         }
26         swap(arr_3020, mid_3020, high_3020);
27     }
28
29     static int partition(int[] arr_3020, int low_3020, int high_3020)
30     {
31         // Panggil fungsi medianOfThree sebelum menentukan pivot
32         medianOfThree(arr_3020, low_3020, high_3020);
33
34         int pivot_3020 = arr_3020[high_3020]; // Sekarang arr[high] sudah menjadi pivot
35         int i_3020 = (low_3020 - 1);
36
37         for (int j_3020 = low_3020; j_3020 <= high_3020 - 1; j_3020++) {
38             // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
39             if (arr_3020[j_3020] < pivot_3020) {
40                 // Increment indeks yang lebih kecil
41                 i_3020++;
42                 swap(arr_3020, i_3020, j_3020);
43             }
44         }
45         swap(arr_3020, i_3020 + 1, high_3020);
46
47         return (i_3020 + 1);
48     }
49
50     static void quickSort_2511533020(int[] arr_3020, int low_3020, int high_3020)
51     {
52         if (low_3020 < high_3020) {
53             int pi_3020 = partition(arr_3020, low_3020, high_3020);
54             quickSort_2511533020(arr_3020, low_3020, pi_3020 - 1);
55             quickSort_2511533020(arr_3020, pi_3020 + 1, high_3020);
56         }
57     }
58
59     public static void printArr_2511533020(int[] arr_3020)
60     {
61         for (int i_3020 = 0; i_3020 < arr_3020.length; i_3020++) {
62             System.out.print(arr_3020[i_3020] + " ");
63         }
64         System.out.println();
65     }
66
67     public static void main(String[] args) {
68         int[] arr_3020 = { 10, 7, 8, 1, 5 };
69         int N_3020 = arr_3020.length;
70         System.out.print("Data sebelum diurutkan: ");
71         printArr_2511533020(arr_3020);
72
73         quickSort_2511533020(arr_3020, 0, N_3020 - 1);
74
75         System.out.print("Data Terurut quickSort: ");
76         printArr_2511533020(arr_3020);
77     }
78 }

```

(Gambar 2.2.4.1)

Kelas QuickSort_2511533020 mengimplementasikan Quick Sort dengan peningkatan median-of-three untuk memilih pivot. Method swap() menukar dua elemen. Method medianOfThree() mengurutkan elemen low, mid, high lalu menukar median ke posisi high sebagai pivot. Method partition() menggunakan pivot dari high, membandingkan elemen dan menukar jika lebih kecil, lalu menempatkan pivot di posisi yang benar. Method

quickSort_2511533020() memanggil dirinya sendiri secara rekursif. Pada method main, array {10,7,8,1,5} diurutkan dan dicetak. Output menunjukkan array terurut ascending..



```

<terminated> QuickSort_2511533020 [Java Applic
Data sebelum diurutkan: 10 7 8 1 5
Data Terurut quickSort: 1 5 7 8 10

```

2.2.5 ShellSort_2511533020

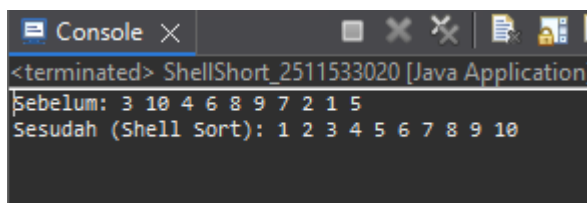
```

1 package pekan8_2511533020;
2
3 public class ShellSort_2511533020 {
4
5     public static void shellSort_2511533020 (int[] A_3020) {
6         int n_3020 = A_3020.length;
7         int gap_3020 = n_3020 / 2;
8         while (gap_3020 > 0) {
9             for (int i_3020 = gap_3020; i_3020 < n_3020; i_3020++) {
10                 int temp_3020 = A_3020[i_3020];
11                 int j_3020 = i_3020;
12                 while (j_3020 >= gap_3020 && A_3020[j_3020 - gap_3020] > temp_3020) {
13                     A_3020[j_3020] = A_3020[j_3020 - gap_3020];
14                     j_3020 = j_3020 - gap_3020;
15                 }
16                 A_3020[j_3020] = temp_3020;
17             }
18             gap_3020 = gap_3020 / 2;
19         }
20     }
21
22     public static void main(String[] args) {
23         int[] data = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
24         System.out.print("Sebelum: ");
25         printArray(data);
26
27         shellSort_2511533020(data);
28
29         System.out.print("Sesudah (Shell Sort): ");
30         printArray(data);
31     }
32
33     public static void printArray(int[] arr) {
34         for (int i_3020 : arr) System.out.print(i_3020 + " ");
35         System.out.println();
36     }
37 }

```

(Gambar 2.2.5.1)

Kelas ShellSort_2511533020 mengimplementasikan Shell Sort. Method shellSort_2511533020() menggunakan gap awal $n/2$, kemudian memperkecil gap hingga 1. Di dalam loop, dilakukan insertion sort dengan jarak gap. Elemen dibandingkan dan digeser ke kanan jika lebih besar. Proses diulang untuk setiap gap. Pada method main, array {3,10,4,6,8,9,7,2,1,5} diurutkan dan dicetak sebelum dan sesudah sorting. Output menunjukkan array terurut ascending.



```

<terminated> ShellSort_2511533020 [Java Application
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, keempat algoritma sorting (Bubble Sort, Merge Sort, Quick Sort, Shell Sort) berhasil diimplementasikan dalam bahasa Java. Bubble Sort dan Merge Sort dilengkapi dengan program GUI untuk visualisasi langkah per langkah sehingga memudahkan pemahaman alur algoritma.

Merge Sort dan Quick Sort menggunakan pendekatan divide and conquer, sedangkan Shell Sort merupakan peningkatan dari Insertion Sort. Semua kode mengikuti aturan penamaan dengan akhiran NIM 3020 pada setiap variabel dan method.

3.2 Saran

Untuk pengembangan lebih lanjut, sebaiknya program GUI Bubble Sort dan Merge Sort dapat dilengkapi dengan fitur pemilihan kecepatan langkah (delay) atau tombol “Auto Run” agar proses sorting berjalan otomatis.

Selain itu, bisa ditambahkan visualisasi untuk Quick Sort dan Shell Sort agar dapat diamati secara interaktif. Pada program console, sebaiknya ditambahkan pencetakan langkah-langkah sorting untuk melihat proses per antara.

3.3 Penutup

Demikian laporan praktikum Algoritma Sorting ini disusun. Laporan ini mencakup penjelasan mengenai Bubble Sort, Merge Sort, Quick Sort, dan Shell Sort beserta implementasinya dalam bentuk console dan GUI.

Semoga laporan ini bermanfaat. Penulis menyadari masih banyak kekurangan, oleh karena itu kritik dan saran yang membangun sangat diharapkan.

DAFTAR PUSTAKA

- [1] Oracle, "Arrays (Java Platform SE 17)," Oracle. [Online]. Available: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Arrays.html> (diakses: 21 Mei 2026).
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
- [3] Wikipedia, "Sorting algorithm," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Sorting_algorithm (diakses: 21 Mei 2026).
- [4] W3Schools, "Java Sorting Algorithms," W3Schools. [Online]. Available: https://www.w3schools.com/java/java_sorting.asp (diakses: 21 Mei 2026).

TUGAS PRAKTIKUM

- **KELAS Lagu_2511533020**

```
• package pekan8_2511533020;
•
• public class Lagu_2511533020 {
•     private String judul_3020;
•     private String penyanyi_3020;
•     private int durasi_3020; // dalam detik
•
•     public Lagu_2511533020(String judul_3020, String penyanyi_3020, int
durasi_3020) {
•         this.judul_3020 = judul_3020;
•         this.penyanyi_3020 = penyanyi_3020;
•         this.durasi_3020 = durasi_3020;
•     }
•
•     public String getJudul_3020() { return judul_3020; }
•     public String getPenyanyi_3020() { return penyanyi_3020; }
•     public int getDurasi_3020() { return durasi_3020; }
•
•     public void setJudul_3020(String judul_3020) { this.judul_3020 =
judul_3020; }
•     public void setPenyanyi_3020(String penyanyi_3020) { this.penyanyi_3020 =
penyanyi_3020; }
•     public void setDurasi_3020(int durasi_3020) { this.durasi_3020 =
durasi_3020; }
•
•     @Override
•     public String toString() {
•         return judul_3020 + " - " + penyanyi_3020 + " (" + durasi_3020 + "
detik)";
•     }
• }
```

Dari kode kelas ini, terlihat bahwa objek lagu disimpan dengan tiga atribut: judul_3020, penyanyi_3020, dan durasi_3020. Semua atribut menggunakan akhiran 3020 sesuai aturan penamaan. Ada konstruktor untuk memudahkan pembuatan objek sekaligus mengisi atribut.

Untuk mengambil data, disediakan getter (getJudul_3020(), getPenyanyi_3020(), getDurasi_3020()). Untuk mengubah, ada setter meskipun di program ini tidak dipakai. Method toString() dicetak ulang agar tampilan lagu menjadi judul - penyanyi (durasi detik) saat diprint.

- **KELAS SORTING_2511533020**

```
• package pekan8_2511533020;
•
• import java.util.Scanner;
•
• public class Sorting_2511533020 {
•     private Lagu_2511533020[] dataLagu_3020;
•     private int jumlahLagu_3020;
•
•     public Sorting_2511533020() {
•         dataLagu_3020 = new Lagu_2511533020[20];
•         jumlahLagu_3020 = 0;
•     }
•
•     // Method inputData_3020 - mengisi data awal minimal 7 lagu (LAGU
•     BERBEDA)
•     public void inputData_3020() {
•         dataLagu_3020[0] = new Lagu_2511533020("Bohemian Rhapsody",
• "Queen", 354);
•         dataLagu_3020[1] = new Lagu_2511533020("Shape of You", "Ed
• Sheeran", 233);
•         dataLagu_3020[2] = new Lagu_2511533020("Blinding Lights", "The
• Weeknd", 200);
•         dataLagu_3020[3] = new Lagu_2511533020("Rolling in the Deep",
• "Adele", 228);
•         dataLagu_3020[4] = new Lagu_2511533020("Uptown Funk", "Bruno
• Mars", 270);
•         dataLagu_3020[5] = new Lagu_2511533020("Hello", "Adele", 295);
•         dataLagu_3020[6] = new Lagu_2511533020("Happier", "Marshmello",
• 204);
•         jumlahLagu_3020 = 7;
•         System.out.println("Data awal (7 lagu) telah dimasukkan.");
•     }
•
•     // Method quickSort_3020 (sama seperti sebelumnya)
•     public void quickSort_3020(int low, int high) {
•         if (low < high) {
•             int pi = partition_3020(low, high);
•             quickSort_3020(low, pi - 1);
•             quickSort_3020(pi + 1, high);
•         }
•     }
•
•     private int partition_3020(int low, int high) {
•         int pivot = dataLagu_3020[high].getDurasi_3020();
•         int i = low - 1;
•         for (int j = low; j < high; j++) {
•             if (dataLagu_3020[j].getDurasi_3020() <= pivot) {
•                 i++;
•                 swap_3020(i, j);
•             }
•         }
•         swap_3020(i + 1, high);
•         return i + 1;
•     }
• }
```

```

    }

    private void swap_3020(int i, int j) {
        Lagu_2511533020 temp = dataLagu_3020[i];
        dataLagu_3020[i] = dataLagu_3020[j];
        dataLagu_3020[j] = temp;
    }

    public void tampilData_3020() {
        if (jumlahLagu_3020 == 0) {
            System.out.println("Tidak ada data lagu.");
            return;
        }
        for (int i = 0; i < jumlahLagu_3020; i++) {
            System.out.println((i + 1) + ". " +
dataLagu_3020[i].toString());
        }
    }

    public void pilihAlgoritma_3020(int pilihan) {
        if (pilihan == 2) {
            System.out.println("\nMemilih Quick Sort (berdasarkan Durasi
Ascending)");
            quickSort_3020(0, jumlahLagu_3020 - 1);
            System.out.println("Data Setelah Quick Sort (Durasi Asc):");
        } else {
            System.out.println("Pilihan tidak valid. Hanya Quick Sort
(2) yang tersedia.");
        }
    }

    public static void main(String[] args) {
        Scanner input_3020 = new Scanner(System.in);
        Sorting_2511533020 program = new Sorting_2511533020();

        System.out.println("=== Sorting Playlist NIM: 2511533020 ===");
        System.out.print("Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
");

        int pilih = input_3020.nextInt();

        program.inputData_3020();

        System.out.println("\nData Sebelum Sorting:");
        program.tampilData_3020();

        program.pilihAlgoritma_3020(pilih);
        program.tampilData_3020();

        input_3020.close();
    }
}

```

Pada kelas `Sorting_2511533020`, disiapkan sebuah array bernama `dataLagu_3020` dengan ukuran 20 untuk menyimpan objek lagu. Variabel `jumlahLagu_3020` digunakan untuk mencatat banyaknya lagu yang telah dimasukkan. Method `inputData_3020()` mengisi array dengan 7 lagu pilihan yang berbeda dari contoh soal, antara lain lagu dari Queen, Ed Sheeran, The Weeknd, Adele, Bruno Mars, dan Marshmello. Durasi lagu dibuat bervariasi (200–354 detik) agar proses sorting dapat diamati dengan jelas.

Method `quickSort_3020()` dan `partition_3020()` diimplementasikan sesuai algoritma Quick Sort. Parameter yang dibandingkan adalah durasi lagu, bukan judul. Pivot diambil dari elemen terakhir, lalu elemen dengan durasi lebih kecil atau sama dipindahkan ke sisi kiri. Method `swap_3020()` digunakan untuk menukar posisi dua elemen dalam array. Proses sorting dilakukan secara rekursif hingga seluruh array terurut.

Method `tampilData_3020()` menampilkan seluruh data lagu yang tersimpan, lengkap dengan nomor urut. Method `pilihAlgoritma_3020()` hanya merespon pilihan 2 (Quick Sort) dan langsung menjalankan `quickSort_3020()`, kemudian mencetak header hasil sorting. Pada method `main`, pengguna diminta memilih angka, lalu data ditampilkan sebelum sorting, diurutkan, dan ditampilkan kembali. Hasil akhir menunjukkan lagu dengan durasi terpendek berada di urutan pertama (Blinding Lights 200 detik) hingga terpanjang di urutan terakhir (Bohemian Rhapsody 354 detik)